1. Create the Controller

👉 File path:

```
src/Controller/ProductController.php
```

```php
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\DataObject\Product\Listing;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(): Response
    {
        // Fetch all Products
        $products = new Listing();
        $products->setOrderKey('name'); // optional: order by name
        $products->setOrder('asc');

        return $this->render('product/list.html.twig', [
            'products' => $products,
        ]);
    }
}
```

---

2. Create the Twig Template

👉 File path:

```
templates/product/list.html.twig
```

```twig
{% extends "base.html.twig" %}

{% block title %}Product Catalog{% endblock %}

{% block content %}
<h1>Product Catalog</h1>

<ul>
    {% for product in products %}
        <li>
            <strong>{{ product.name }}</strong> - ${{ product.price }}
        </li>
    {% else %}
        <li>No products found.</li>
    {% endfor %}
</ul>
{% endblock %}
```

---

3. Create a Document in Pimcore Admin

1. Go to **Documents** → **Create Page** → **Name it** `products`
2. Open page → **Settings tab** → set:
    - **Controller**: `App\Controller\ProductController::list`
    - **Template**: `product/list.html.twig`
3. Publish ✅

---

4. Test it

Visit in browser:

```
http://yourdomain.local/products
```

You should now see a simple product list with **name + price**.

---

1. Add Method to Controller

👉 File: `src/Controller/ProductController.php`

```php
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(): Response
    {
        $products = new Product\Listing();
        $products->setOrderKey('name');
        $products->setOrder('asc');

        return $this->render('product/list.html.twig', [
            'products' => $products,
        ]);
    }

    #[Route('/product/{id}', name: 'product_detail')]
    public function detail(int $id): Response
    {
        // Load Product by ID
        $product = Product::getById($id);

        if (!$product) {
            throw $this->createNotFoundException("Product with ID $id not
found.");
        }

        return $this->render('product/detail.html.twig', [
            'product' => $product,
        ]);
    }
}
```

---

2. Create Detail Twig Template

👉 File: `templates/product/detail.html.twig`

```twig
{% extends "base.html.twig" %}

{% block title %}{{ product.name }}{% endblock %}

{% block content %}
<h1>{{ product.name }}</h1>

<p><strong>Price:</strong> ${{ product.price }}</p>

<div>
    <strong>Description:</strong>
    {{ product.description|raw }}
</div>

{% if product.image %}
    <div>
        <img src="{{ product.image.thumbnail('galleryThumbnail') }}" alt="{{
product.name }}">
    </div>
{% endif %}

<a href="{{ path('product_list') }}">← Back to catalog</a>
{% endblock %}
```

---

3. Add Links from Product List

👉 Update `templates/product/list.html.twig`

```twig
<ul>
    {% for product in products %}
        <li>
            <a href="{{ path('product_detail', {id: product.id}) }}">
                <strong>{{ product.name }}</strong>
            </a>
            - ${{ product.price }}
        </li>
    {% else %}
        <li>No products found.</li>
    {% endfor %}
</ul>
```

---

4. Test in Browser

- Product list:

```
http://yourdomain.local/products
```

- Product detail:

```
http://yourdomain.local/product/123
```

(replace `123` with your product's real ID from Pimcore admin).

⚡ Now you've got:

- A **list page** → all products
- A **detail page** → each product with description + image

---

1. Update Controller

👉 File: `src/Controller/ProductController.php`

```php
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\DataObject\Category;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(): Response
    {
        $products = new Product\Listing();
        $products->setOrderKey('name');
        $products->setOrder('asc');

        return $this->render('product/list.html.twig', [
            'products' => $products,
        ]);
```

```
    }

    #[Route('/product/{id}', name: 'product_detail')]
    public function detail(int $id): Response
    {
        $product = Product::getById($id);

        if (!$product) {
            throw $this->createNotFoundException("Product with ID $id not
found.");
        }

        return $this->render('product/detail.html.twig', [
            'product' => $product,
        ]);
    }

    #[Route('/category/{id}', name: 'products_by_category')]
    public function byCategory(int $id): Response
    {
        $category = Category::getById($id);

        if (!$category) {
            throw $this->createNotFoundException("Category with ID $id not
found.");
        }

        $products = new Product\Listing();
        $products->setCondition('category__id = ?', $category->getId());
        $products->setOrderKey('name');
        $products->setOrder('asc');

        return $this->render('product/by_category.html.twig', [
            'category' => $category,
            'products' => $products,
        ]);
    }
}
```

---

2. Create Twig Template

👉 File: `templates/product/by_category.html.twig`

```
{% extends "base.html.twig" %}

{% block title %}Products in {{ category.name }}{% endblock %}

{% block content %}
<h2>{{ category.name }}</h2>
```

```
{% if category.description %}
    <p>{{ category.description }}</p>
{% endif %}

<ul>
    {% for product in products %}
        <li>
            <a href="{{ path('product_detail', {id: product.id}) }}">
                <strong>{{ product.name }}</strong>
            </a>
            - ${{ product.price }}
        </li>
    {% else %}
        <li>No products found in this category.</li>
    {% endfor %}
</ul>

<a href="{{ path('product_list') }}">← Back to all products</a>
{% endblock %}
```

---

3. Test in Browser

- Visit:

  ```
  http://yourdomain.local/category/12
  ```

  (replace `12` with the real **Category ID** from Pimcore admin).

- You should see the category name, its description, and only products linked to it.

✅ Now you have:

- `/products` → all products
- `/product/{id}` → product detail
- `/category/{id}` → category-filtered product list

---

1. Update Controller

👉 File: `src/Controller/ProductController.php`

```php
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Pimcore\Model\DataObject\Product\Listing;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products/search', name: 'product_search')]
    public function search(Request $request): Response
    {
        $query = $request->query->get('q'); // ?q=searchterm
        $products = [];

        if ($query) {
            $products = new Listing();
            $products->setCondition('name LIKE ?', ["%$query%"]);
            $products->setOrderKey('name');
            $products->setOrder('asc');
        }

        return $this->render('product/search.html.twig', [
            'products' => $products,
            'query'    => $query,
        ]);
    }
}
```

---

2. Create Twig Template

👉 File: `templates/product/search.html.twig`

```twig
{% extends "base.html.twig" %}

{% block title %}Product Search{% endblock %}

{% block content %}
<h1>Search Products</h1>

<form method="get" action="{{ path('product_search') }}">
    <input type="text" name="q" value="{{ query }}" placeholder="Search by
name">
    <button type="submit">Search</button>
</form>
```

```
{% if query %}
    <h2>Results for "{{ query }}"</h2>

    <ul>
        {% for product in products %}
            <li>
                <a href="{{ path('product_detail', {id: product.id}) }}">
                    <strong>{{ product.name }}</strong>
                </a>
                - ${{ product.price }}
            </li>
        {% else %}
            <li>No products found.</li>
        {% endfor %}
    </ul>
{% endif %}
{% endblock %}
```

---

3. Test in Browser

Go to:

```
http://yourdomain.local/products/search?q=phone
```

- The form will appear at `/products/search`.
- Enter a product name → results render dynamically.

✅ Now you have a **live product search** using Pimcore objects.

---

1. Update Controller

👉 File: `src/Controller/ProductController.php`

```
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product\Listing;
```

```php
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Request;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products', name: 'product_list')]
    public function list(Request $request): Response
    {
        $page = max(1, (int) $request->query->get('page', 1)); // current
page
        $perPage = 10; // items per page
        $offset = ($page - 1) * $perPage;

        // Fetch paginated products
        $products = new Listing();
        $products->setLimit($perPage);
        $products->setOffset($offset);
        $products->setOrderKey('name');
        $products->setOrder('asc');

        // Total count
        $totalProducts = $products->getTotalCount();
        $totalPages = ceil($totalProducts / $perPage);

        return $this->render('product/list.html.twig', [
            'products'     => $products,
            'page'         => $page,
            'totalPages'   => $totalPages,
        ]);
    }
}
```

---

2. Update Twig Template

👉 File: `templates/product/list.html.twig`

```twig

{% extends "base.html.twig" %}

{% block title %}Product Catalog{% endblock %}

{% block content %}
<h1>Product Catalog</h1>

<ul>
    {% for product in products %}
        <li>
            <a href="{{ path('product_detail', {id: product.id}) }}">
                <strong>{{ product.name }}</strong>
```

```
            </a>
            - ${{ product.price }}
        </li>
    {% else %}
        <li>No products found.</li>
    {% endfor %}
</ul>

<!-- Pagination -->
<div class="pagination">
    {% if page > 1 %}
        <a href="{{ path('product_list', {page: page - 1}) }}">←
Previous</a>
    {% endif %}

    <span>Page {{ page }} of {{ totalPages }}</span>

    {% if page < totalPages %}
        <a href="{{ path('product_list', {page: page + 1}) }}">Next →</a>
    {% endif %}
</div>
{% endblock %}
```

---

3. Test in Browser

Go to:

```
http://yourdomain.local/products?page=1
```

Click **Next** → loads page 2, etc.

✅ Now your product list is paginated (10 per page).

---

1. Controller

👉 `src/Controller/ProductController.php`

```
<?php

namespace App\Controller;
```

```php
use Pimcore\Model\DataObject\Product\Listing;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/products/sort/{field}', name: 'product_sort')]
    public function sort(string $field = 'name'): Response
    {
        // allow only safe fields
        $allowedFields = ['name', 'price'];
        if (!in_array($field, $allowedFields, true)) {
            $field = 'name';
        }

        $products = new Listing();
        $products->setOrderKey($field);
        $products->setOrder('asc');

        return $this->render('product/sort.html.twig', [
            'products' => $products,
            'field'    => $field,
        ]);
    }
}
```

---

2. Twig Template

👉 `templates/product/sort.html.twig`

```twig
{% extends "base.html.twig" %}

{% block title %}Products sorted by {{ field }}{% endblock %}

{% block content %}
<h1>Products (sorted by {{ field }})</h1>

<p>
    Sort by:
    <a href="{{ path('product_sort', {field: 'name'}) }}">Name</a> |
    <a href="{{ path('product_sort', {field: 'price'}) }}">Price</a>
</p>

<ul>
    {% for product in products %}
        <li>
            <strong>{{ product.name }}</strong> - ${{ product.price }}
        </li>
```

```
    {% else %}
        <li>No products found.</li>
    {% endfor %}
</ul>
{% endblock %}
```

---

3. Test in Browser

- By Name:

  ```
  http://yourdomain.local/products/sort/name
  ```

- By Price:

  ```
  http://yourdomain.local/products/sort/price
  ```

---

1. Controller Update

👉 src/Controller/ProductController.php

```php
<?php

namespace App\Controller;

use Pimcore\Model\DataObject\Product;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\Response;
use Symfony\Component\Routing\Annotation\Route;

class ProductController extends AbstractController
{
    #[Route('/product/{id}', name: 'product_detail')]
    public function detail(int $id): Response
    {
        $product = Product::getById($id);

        if (!$product) {
            throw $this->createNotFoundException("Product not found");
        }
```

```
        $category = $product->getCategory();

        return $this->render('product/detail.html.twig', [
            'product'  => $product,
            'category' => $category,
        ]);
    }
}
```

---

2. Twig Template

👉 `templates/product/detail.html.twig`

```
{% extends "base.html.twig" %}

{% block title %}{{ product.name }}{% endblock %}

{% block content %}
<h1>{{ product.name }}</h1>

<p><strong>Price:</strong> ${{ product.price }}</p>
<p><strong>Description:</strong> {{ product.description|raw }}</p>

{% if product.image %}
    <div>
        <img src="{{ product.image.getThumbnail('galleryThumbnail') }}"
alt="{{ product.name }}">
    </div>
{% endif %}

{% if category %}
    <hr>
    <h2>Category: {{ category.name }}</h2>
    <p>{{ category.description }}</p>
{% endif %}

<a href="{{ path('product_sort', {field: 'name'}) }}">Back to Products</a>
{% endblock %}
```

✅ Now each **Product detail page** will show:

- Name, Price, Description, Image
- Related **Category Name + Description**

Got it ✅ Instead of a `<ul>`, let's render your **Product List** in a nice **responsive grid of cards** using **TailwindCSS** (same structure works with Bootstrap too, I'll note both).

---

1. Controller (no change needed)

Assume you already have:

```
#[Route('/products', name: 'product_list')]
public function list(): Response
{
    $products = new \Pimcore\Model\DataObject\Product\Listing();

    return $this->render('product/list.html.twig', [
        'products' => $products,
    ]);
}
```

---

2. TailwindCSS Card Grid

👉 templates/product/list.html.twig

```
{% extends "base.html.twig" %}

{% block title %}Product Catalog{% endblock %}

{% block content %}
<h1 class="text-2xl font-bold mb-6">Product Catalog</h1>

<div class="grid grid-cols-1 sm:grid-cols-2 md:grid-cols-3 lg:grid-cols-4
gap-6">
    {% for product in products %}
        <div class="bg-white rounded-2xl shadow-md overflow-hidden
hover:shadow-lg transition">
            {% if product.image %}
                <img src="{{ product.image.getThumbnail('galleryThumbnail')
}}"
                     alt="{{ product.name }}"
                     class="w-full h-48 object-cover">
            {% else %}
                <div class="w-full h-48 bg-gray-200 flex items-center
justify-center text-gray-500">
                    No Image
                </div>
```

```
            {% endif %}

            <div class="p-4">
                <h2 class="text-lg font-semibold">{{ product.name }}</h2>
                <p class="text-gray-600">${{ product.price }}</p>

                {% if product.category %}
                    <p class="text-sm text-gray-500 mt-1">
                        Category: <span class="font-medium">{{
product.category.name }}</span>
                    </p>
                {% endif %}

                <a href="{{ path('product_detail', {id: product.id}) }}"
                   class="mt-3 inline-block bg-blue-600 text-white px-3 py-1
rounded-lg text-sm hover:bg-blue-700">
                    View Details
                </a>
            </div>
        </div>
    {% endfor %}
</div>
{% endblock %}
```

---

3. If You Prefer Bootstrap

```
<div class="row">
    {% for product in products %}
        <div class="col-md-3 mb-4">
            <div class="card h-100 shadow-sm">
                {% if product.image %}
                    <img src="{{
product.image.getThumbnail('galleryThumbnail') }}"
                         class="card-img-top"
                         alt="{{ product.name }}">
                {% endif %}

                <div class="card-body">
                    <h5 class="card-title">{{ product.name }}</h5>
                    <p class="card-text">${{ product.price }}</p>
                    {% if product.category %}
                        <small class="text-muted">Category: {{
product.category.name }}</small>
                    {% endif %}
                </div>
                <div class="card-footer">
                    <a href="{{ path('product_detail', {id: product.id}) }}"
class="btn btn-primary btn-sm">
                        View Details
                    </a>
```

```
                </div>
            </div>
        </div>
    {% endfor %}
</div>
```

```
{# Tailwind via CDN (you can also install it properly if needed) #}
<script src="https://cdn.tailwindcss.com"></script>
```

```
<!-- Header -->
<header class="bg-blue-600 text-white shadow-md">
    <div class="container mx-auto px-4 py-4 flex justify-between items-
center">
        <h1 class="text-xl font-bold">
            <a href="{{ path('product_list') }}">🛍️ My Shop</a>
        </h1>
        <nav>
            <a href="{{ path('product_list') }}" class="px-3
hover:underline">Products</a>
            <a href="#" class="px-3 hover:underline">About</a>
            <a href="#" class="px-3 hover:underline">Contact</a>
        </nav>
    </div>
</header>

<!-- Main Content -->
<main class="container mx-auto px-4 py-8">
    {% block content %}{% endblock %}
</main>

<!-- Footer -->
<footer class="bg-gray-800 text-gray-300 text-center py-4 mt-8">
    <p>&copy; {{ "now"|date("Y") }} My Shop. All rights reserved.</p>
</footer>
```

Got it 👍 Let’s make a simple **API endpoint** to return all products as JSON.

---

1. Controller

👉 src/Controller/Api/ProductApiController.php

```php
<?php

namespace App\Controller\Api;

use Pimcore\Model\DataObject\Product;
use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
use Symfony\Component\HttpFoundation\JsonResponse;
use Symfony\Component\Routing\Annotation\Route;

class ProductApiController extends AbstractController
{
    #[Route('/api/products', name: 'api_products', methods: ['GET'])]
    public function index(): JsonResponse
    {
        $listing = new Product\Listing();
        $listing->setOrderKey('name');
        $listing->setOrder('ASC');

        $data = [];

        foreach ($listing as $product) {
            $data[] = [
                'id'          => $product->getId(),
                'name'        => $product->getName(),
                'sku'         => $product->getSku(),
                'price'       => $product->getPrice(),
                'description' => $product->getDescription(),
                'category'    => $product->getCategory()?->getName(),
                'image'       => $product->getImage()
                                    ? $product->getImage()->getFullPath()
                                    : null,
            ];
        }

        return $this->json($data);
    }
}
```

---

2. Test

After clearing cache (`php bin/console cache:clear`), open:

```
http://yourdomain.local/api/products
```

You'll get JSON like:

```
[
  {
    "id": 123,
    "name": "iPhone 16",
    "sku": "IP16",
    "price": 1200,
    "description": "<p>Latest iPhone</p>",
    "category": "Smartphones",
    "image": "/var/assets/products/iphone16.jpg"
  },
  {
    "id": 124,
    "name": "MacBook Pro",
    "sku": "MBP",
    "price": 2500,
    "description": "<p>High performance laptop</p>",
    "category": "Laptops",
    "image": "/var/assets/products/macbookpro.jpg"
  }
]
```